Class 17: Version Control with Git

Data-driven Systems Engineering

Agenda

- Why version control is required for a growing modeling codebase and a team project.
- Git fundamentals: commits, history, and differences between versions.
- Branching and merging, including conflict resolution.
- Remotes: pushing, pulling, and collaborating through GitHub.
- Pull requests and code review as a team practice.
- What must never be committed.

Why version control, and why now

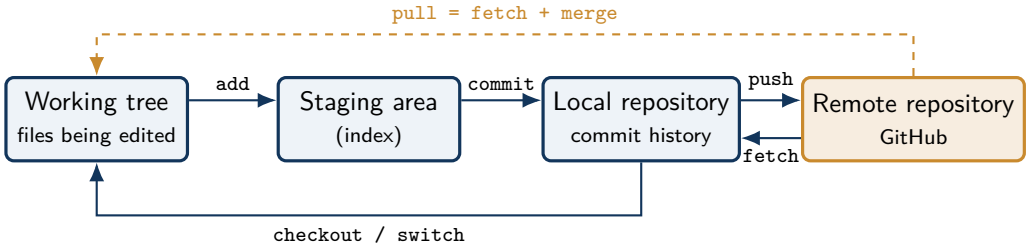
- Every previous class produced code that changed over time: notebooks, a Streamlit application, feature pipelines.
- Without version control, the question "which version produced this result?" cannot be answered reliably.
- Git gives every experiment, threshold choice, and interface change a traceable history; it complements the experiment tracking of Class 18, which records runs but not the code that produced them.
- The practice must be established now, before the team project (Classes 32–34) makes it indispensable.

Git fundamentals

```
git init
git clone <url>
git status
git add <file>
git commit -m "message"
git log --oneline
git diff
```

- A commit is a snapshot of the project together with a message, an author, a timestamp, and a reference to its parent commit(s); it is identified by a hash of this content.
- The commits form a directed acyclic graph: the history. `git log` traverses it, and `git diff` shows the difference between two snapshots.

Where a change lives: the four locations



- `git status` reports the differences between the working tree, the staging area, and the last commit.
- `git diff` compares the working tree with the staging area; `git diff --staged` compares the staging area with the last commit.
- Only committed changes become part of the history; only pushed commits are visible to the team.

What makes a good commit

- **Atomic:** one logical change per commit. **Explanatory:** the message states why; the difference already shows what.
- **Frequent and consistent:** the history documents the evolution of the project, and each commit leaves the code in a working state where practical.

Uninformative message	**Informative message**
`update`	`Use cost-based threshold (0.06) in fraud app`
`fix`	`Fit scaler on training split only to avoid leakage`
`final version`	`Add rolling-mean feature for temperature (w = 10)`
`changes in notebook`	`Replace label-encoded chi2 by test of independence`

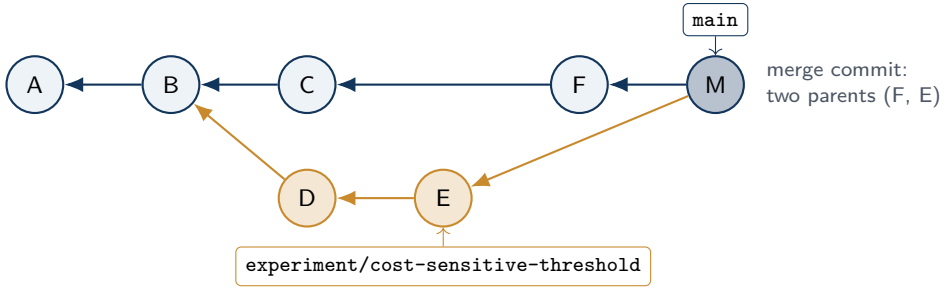
Convention: a short summary line in the imperative mood ($\lesssim 50$ characters), optionally followed by a blank line and a body explaining the motivation.

Branching to test an idea safely

```
git checkout -b experiment/cost-sensitive-threshold
git add .
git commit -m "Try cost-sensitive threshold tuning"
git push -u origin experiment/cost-sensitive-threshold
# compare results, then merge or discard the branch
```

- A branch is a movable reference to a commit; it isolates an exploratory change, so that `main` keeps the last validated model and interface.
- If the experiment does not outperform the baseline, the branch is discarded without affecting `main`.

Branches and merges as a commit graph



- Each commit points to its parent(s); arrows go backwards in time. A branch name is only a reference to the latest commit of that line of work.
- The experiment branched at B; meanwhile `main` advanced to C and F. The merge commit M combines both lines and has two parents.
- Discarding the experiment instead would simply delete the reference `experiment/...`: commits D and E would no longer be reachable from any branch.

Merging and conflicts

```
git checkout main
git merge experiment/cost-sensitive-threshold
# CONFLICT (content): Merge conflict in scoring.py
```

- A conflict arises when both branches modified the same region of a file. Git marks it with <<<<<<<, =======, and >>>>>>>; resolving it requires a decision about the intended behavior.
- The resolution consists of choosing or combining the two versions and committing the result, which becomes a merge commit with two parents.
- Conflicts are an expected part of collaborative development, not an indication of failure.

A conflict in practice: the decision threshold

Conflict in `scoring.py`

```
<<<<<<< HEAD
THRESHOLD = 0.5    # library default
=======
THRESHOLD = 0.06   # cost-based (Class 18)
>>>>>>> experiment/cost-sensitive-threshold
```

Resolution

```
# c_FN = 100 c_FP; selected on the
# validation split (Class 18)
THRESHOLD = 0.06
```

- `HEAD` is the version of the current branch (`main`); the second block comes from the branch being merged.
- Git cannot decide which threshold is correct: the choice is a modeling decision, justified by the cost analysis.
- The resolution removes the markers, documents the reason, and is committed:

```
git add scoring.py
git commit -m "Merge cost-based threshold"
```

Remotes and collaboration through GitHub

```
git remote -v
git push origin main
git pull origin main
git fetch
```

- A remote is a shared copy of the repository; in this course, hosted on GitHub.
- `push` sends local commits to the remote; `fetch` retrieves new remote commits; `pull` performs a fetch followed by a merge into the local branch.
- Pulling before starting new work limits unnecessary divergence from the rest of the team.

Pull requests and code review

- A pull request is a review checkpoint before code reaches a shared branch, where a teammate can detect an unintended change of behavior.
- Small pull requests can be reviewed thoroughly; large ones tend to be approved without real scrutiny.
- Review comments address the code and the decisions behind it, not its author.
- For the team project, a branching convention (e.g. one feature branch per task) must be agreed before the first commit, not after the first conflict.

What must never be committed

```
/.ipynb_checkpoints
/venv
.DS_Store

__pycache__/
*.py[cod]
*.aux
*.log
```

- Excerpt of this repository's `.gitignore`: environments, caches, and build artifacts are regenerated and need no history.
- Secrets and credentials must never be committed: a later commit does not remove them from the history, so a leaked credential must be revoked.
- Large trained artifacts are often better stored outside Git, e.g. in a model registry (Class 30).

Handling secrets correctly

Incorrect: secret in the code

```
run = neptune.init_run(
    project="team/project",
    api_token="a1b2c3d4...",  # leaked
)
```

Correct: secret in the environment

```
import os
token = os.environ["NEPTUNE_API_TOKEN"]
run = neptune.init_run(
    project="team/project",
    api_token=token,
)
```

- The secret is stored outside the repository, e.g. exported in the shell or kept in a local `.env` file listed in `.gitignore`.
- The code documents which variable is required, never its value.
- If a secret has been committed: (1) revoke it immediately, (2) issue a new one, (3) optionally rewrite the history (e.g. `git filter-repo`); once pushed, the secret must be considered compromised regardless of the rewrite.

Guided practice

Work in pairs on a shared GitHub repository (about 30 minutes):

1. Student A creates the repository with a `README.md`, a `.gitignore`, and a file `scoring.py` containing `THRESHOLD = 0.5`; student B clones it.
2. Each student creates a branch: A sets `THRESHOLD = 0.3`, B sets `THRESHOLD = 0.06`, each with an informative commit message.
3. A pushes and merges first through a pull request reviewed by B.
4. B pulls `main`, merges it into the own branch, and resolves the resulting conflict, justifying the chosen value in the commit message.
5. B opens a pull request; A reviews it and merges.
6. Both inspect the result with `git log --oneline --graph` and identify the merge commits.

Deliverable: the URL of the repository; its commit graph must show two branches, one resolved conflict, and two reviewed pull requests.

Repository connections

Repo activity

- The commit history and the `.gitignore` of this repository are a working example of the practices of this class.
- `notebooks/`, `streamlit/`, and `pasteurization/` contain rapidly changing code that benefits from small, frequent commits.
- In the team project (Classes 32–34), each team applies these practices to its own shared repository.

Papers and materials

- Chacon, S., Straub, B. (2014). *Pro Git*, 2nd ed. Apress. git-scm.com/book.
- Driessen, V. (2010). A successful Git branching model.
- Bird, C., Rigby, P. C., Barr, E. T., Hamilton, D. J., German, D. M., Devanbu, P. (2009). The promises and perils of mining Git. *Proceedings of the 6th IEEE International Working Conference on Mining Software Repositories*, 1–10.
- Bacchelli, A., Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 712–721.
- GitHub documentation: pull requests.
- Atlassian Git tutorials: resolving merge conflicts.

Take-away message

Version control makes every result traceable
to the code that produced it:

commit small and often, isolate experiments in branches,
review before merging, and never commit secrets.

Thank you for your attention.

Next: Class 18 — modeling, experiment tracking, evaluation, and decision-making.